

JEE Webshop Dokumentation

Luca Malisan

2025

Einführung und Ziele

Aufgabenstellung

Bei dem zu entwickelnden System handelt es sich um einen Webshop. Dieser besteht aus den folgenden Komponenten:

- K: Digitaler Katalog aller Produkte
- D: Detailansicht jedes produkts
- W: Warenkorb
- L: Login
- R: Rest-API zur Anbindung externer Systeme

Im Produktkatalog müssen Einträge per Volltextsuche gefunden werden können (K1). Zudem muss die Menge der Resultate nach Haupt- und Subkategorie eingeschränkt werden können (K2). Jeder Eintrag in der Liste muss Artikelbild, Titel, Preis, Rabatt und Verfügbarkeit anzeigen (K3). Pro Seite muss die Anzahl Einträge limitiert werden, Navigation zwischen den Seiten muss möglich sein (K4). Durch Klick auf einen Eintrag muss die Detailseite geöffnet werden. (K5)

Auf der Detailansicht müssen zusätzlich zum Listeneintrag alle anderen Bilder und die Beschreibung sowie die verfügbare Anzahl angezeigt werden (D1). Zudem muss ein Button angezeigt werden, um den Artikel zum Warenkorb zu hinzufügen. Die Anzahl muss hierbei einstellbar sein (D2).

Der Warenkorb muss einen Überblick über alle hinzugefügten Artikel bieten (W1). Die Anzahl jedes Artikels muss angepasst werden können bzw. Artikel müssen entfernt werden können (W2). Der Gesamtpreis mit und ohne Mwst. sowie Rabatte müssen ebenfalls angezeigt werden (W3).

Benutzer müssen über einen Login-Button auf die Login-Page gelangen (L1). Unautorisierte Benutzer können alle Artikel ansehen, um etwas zum Warenkorb hinzuzufügen wird aber ein Login verlangt (L2). Das Login muss dabei in einem separaten Login-Formular stattfinden (L3). Die eingegebene E-Mail muss durch eine Bestätigungsmail validiert werden (L4).

Für das Hinzufügen, Updaten und Löschen von Artikeln und Kategorien muss eine REST-API zur Verfügung gestellt werden (R1). Diese darf nur von authentifizierten Maschinen angesteuert werden (R2).

Qualitätsziele

- Für eine gute Usability muss der Webshop spätestens zwei Wochen vor einem neuen Versionrelease in Usability-Tests mit Testkunden mindestens 90% Zufriedenheit erreichen
- Um eine optimale Performance zu garantieren, darf jede Operation im Webshop (Website laden, Artikel zum Warenkorb hinzufügen, ...) maximal 500ms dauern, sofern die CPU-Auslastung unter 90% liegt.
- Die Applikation muss mindestens 99.9% der Zeit vollständig verfügbar sein.
- Um Partitionstoleranz zwischen den Komponenten zu gewährleisten, Innerhalb eines Monitoring-Zeitraums von 7 Tagen dürfen max. 0.1% der Exceptions zu einem Absturz führen.
- Für die fachliche Korrektheit müssen alle Methoden, die Business-Logik bzw. Berechnungen durchführen, mit mindestens fünf Unit-Tests abgesichert werden. Generell werden mindestens 80% Testabdeckung angestrebt.
- Sämtliche der verwendeten Dependencies dürfen keine bekannten / dokumentierten Sicherheitslücken aufweisen. Vor jedem Versionsrelease muss deshalb eine solche Sicherheitsprüfung durchgeführt werden.
- Für die Verständlichkeit des Codes müssen sämtliche Methoden und REST-Schnittstellen über Javadoc korrekt dokumentiert sein.
- Bei den redundanten Daten im Webshop, die aus angebundenen Systemen stammen, muss deren Ursprung bei der Speicherung in der Datenbank als Metadaten angegeben werden.

Vision

Unsere Vision basiert auf einem Webshop, der als technisch souveräne Lösung vermarktet wird, die eigenständig und technologieunabhängig betrieben werden kann. Dank der Verwendung einer eigenen Datenbank garantieren wir eine nahtlose Integration in bestehende IT-Infrastrukturen, ohne dass eine Hinzunahme weiterer Produkte von Drittanbietern nötig ist.

Der Webshop kommt mit einer kleinen Anzahl von Abhängigkeiten aus, womit Wartungen mit maximaler Effizienz durchgeführt werden können. Zudem werden so der Anpassbarkeit keine Grenzen gesetzt, kein Kundenwunsch ist uns zu komplex. Diese Kombination erlaubt uns, auch hochspezialisierte Kundenprojekte zu attraktiven Konditionen umzusetzen. Damit weitet sich unsere Marktzielgruppe auf Unternehmen aller Art und Grösse aus - sei es ein Startup, ein traditionelles Familienunternehmen oder ein Industriekonzern. Durch eine hohe Quantität durchgeführter Projekte wird unsere Lösung regelmässig validiert. Die hohe Anzahl Testimonials gibt potentiellen Neukunden ein Sicherheitsgefühl und dient als Vertrauensbeweis.

In der heutigen Welt werden Systeme technisch immer komplexer und richten sich in erster Linie nach den aktuellsten Technologietrends. Hierbei werden allzu oft die tatsächlichen Kundenanforderungen missachtet. Oft zahlen Unternehmen hohe Preise für Funktionen, die sie gar nicht brauchen. Mit unserem minimalistischen und flexiblen Ansatz berufen wir uns auf traditionelle, fundierte Werte der Software-Entwicklung. Bei uns steht der Kunde im Fokus, nicht die Tech-Industrie.

Kontextabgrenzung

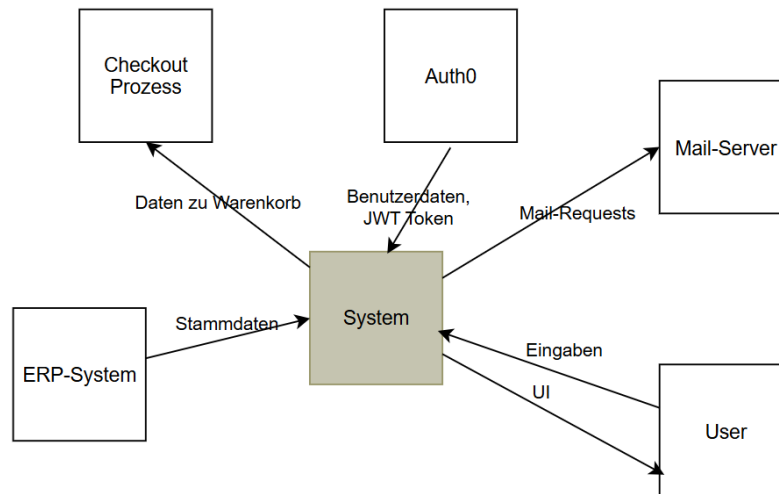


Figure 1: Technischer Kontext

Lösungsstrategie

Inhalt

Für die Entwicklung des Systems wurden die folgenden zentralen Entscheidungen getroffen:

- Da das Projekt aus Sicht der Funktionalität relativ schlank ist, soll die Projektgröße auf das nötige Minimum reduziert werden. So kann eine niedrige Komplexität und eine hohe Performance erreicht werden.
- Um die korrekte Funktionalität der Applikation sicherzustellen, muss die Business-Logik nach jeder Änderung getestet werden.
- Um Sicherheitslücken zu verhindern und den Datenschutz einzuhalten, müssen Zugriffe auf Daten zwingend autorisiert werden. Benutzer dürfen nur auf öffentliche und ihre eigenen Daten zugreifen, Änderungen an den Stammdaten müssen durch berechtigte Applikationen erfolgen.
- Login- und Checkout-Prozess des Webshops sind hinsichtlich Datenschutz und Sicherheit am kritischsten, weshalb diese an eine externe Software delegiert wird. Es gibt hierfür bereits umfangreiche, standardisierte Lösungen.

Bausteinsicht

Das System besteht aus den folgenden physischen und logischen Komponenten:

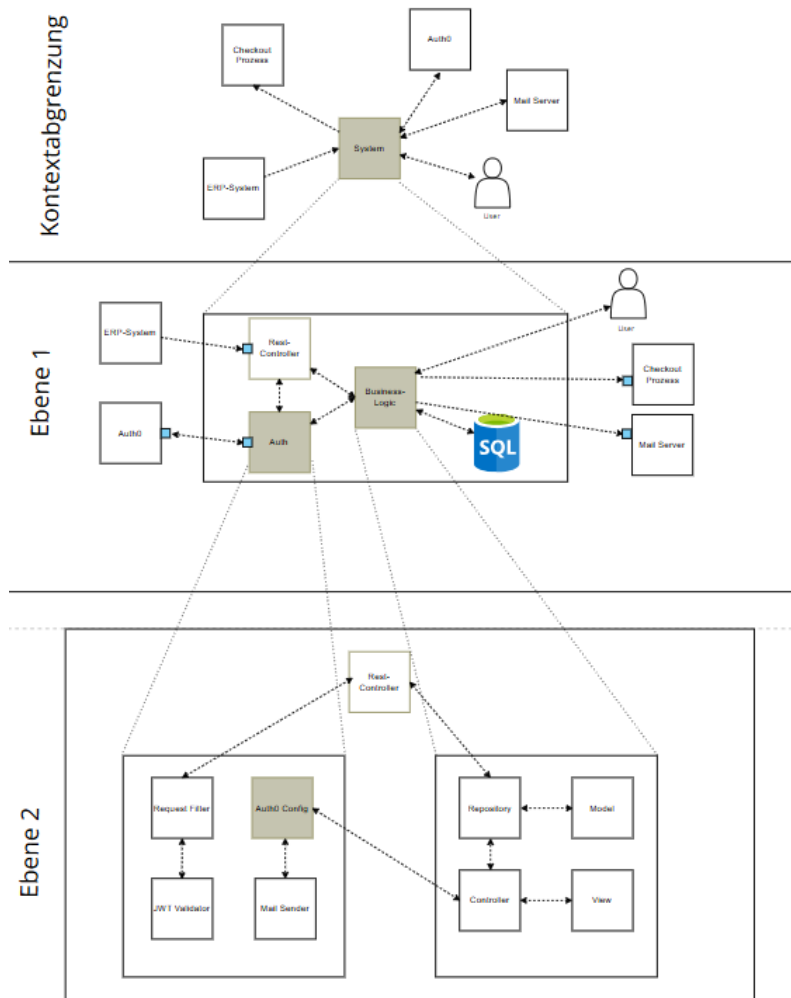


Figure 2: Systemaufbau

Kontextabgrenzung

- ERP-System: Im ERP-System werden die Artikeldaten und Kategorien verwaltet. Die Daten werden bei Änderungen in den Webshop propagiert, um die Konsistenz der Daten sicherzustellen.
- Checkout Prozess: Der Kunde wird vom Webshop aus auf das Checkout-System weitergeleitet, um seine Bestellung abzuschliessen.
- Auth0: Der Kunde wird vom Webshop auf den Login-Screen von Auth0 weitergeleitet. Nach erfolgreicher Eingabe seiner Daten gelangt er wieder zurück und die Daten werden verarbeitet.
- Der Benutzer interagiert mit dem GUI des Webshops, dieser verarbeitet die Eingaben und aktualisiert die Darstellung entsprechend

Ebene 1

- Rest-Komponente: Um Daten aus angebundenen Systemen wie dem ERP-Software zu erhalten, stellt der Webshop eine REST-Schnittstelle bereit. Darüber können Änderungen an den Daten zu Artikeln und Kategorien durchgeführt werden.
- Auth-Komponente: Diese Komponente kümmert sich um die Authentifizierung und Autorisierung der Zugriffe auf den Webshop und die REST-API.
- Business-Logic-Komponente: Dort wird die eigentliche Funktionalität des Webshops umgesetzt. Die Komponente verarbeitet Benutzereingaben und setzt die Befehle um. Diese bestehen im Wesentlichen aus Änderungen und Darstellungen von Daten im Webshop.

Ebene 2

- Request-Filter: Dieser überprüft jeden einzelnen Request auf die REST-Schnittstellen und stellt sicher, dass der Zugriff autorisiert erfolgt. Ist dies nicht der Fall, wird eine entsprechende Fehlerbehandlung umgesetzt.
- JWT-Validator: Dabei handelt es sich um eine Hilfsklasse, um sicherzustellen, dass ein gegebenes JWT eine valide Struktur besitzt und noch gültig ist.
- Auth0 Flow Handler: Diese Klasse dient als Orchestrator, sie koordiniert die nötigen Aktionen um den ganzen Auth0 Login Flow korrekt umzusetzen.
- Auth0 Config: Diese Komponente besteht aus diversen Hilfsklassen, um Daten zwischenspeichern und den Auth0 Flow abbilden zu können.
- Mail Sender: Dort wird die Logik gekapselt, um ein Bestätigungsmail zu erstellen und an den User zu senden.
- Model - View - Controller: Um die Business-Logik umzusetzen, wird das MVC-Pattern verwendet. Die Struktur der HTML-Seite wird durch die View definiert. Durch klar definierte Listener werden bei Aktionen des Benutzers Events ausgeführt, die vom Controller behandelt werden. Dieser verwaltet einerseits temporäre Variablen, aber koordiniert auch das Rendern der korrekten Views. Zudem hat er Zugriff auf die Datenbank mithilfe von Repositories, die dazu dedizierte Methoden anbieten. Änderungen an der Datenbank werden auf Model-Klassen umgesetzt, die die Tabellen im Code abbilden.

Laufzeitsicht

Login Flow

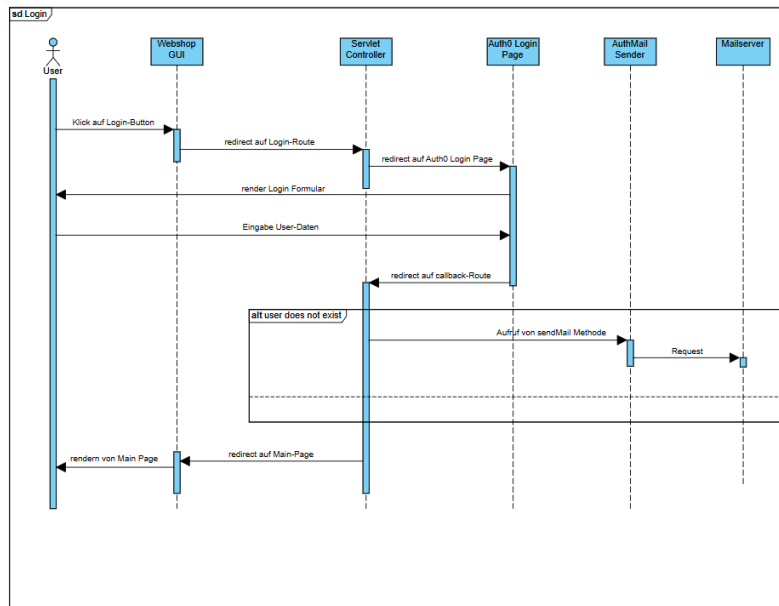


Figure 3: Login-Flow Sequenzdiagramm

REST-API Call

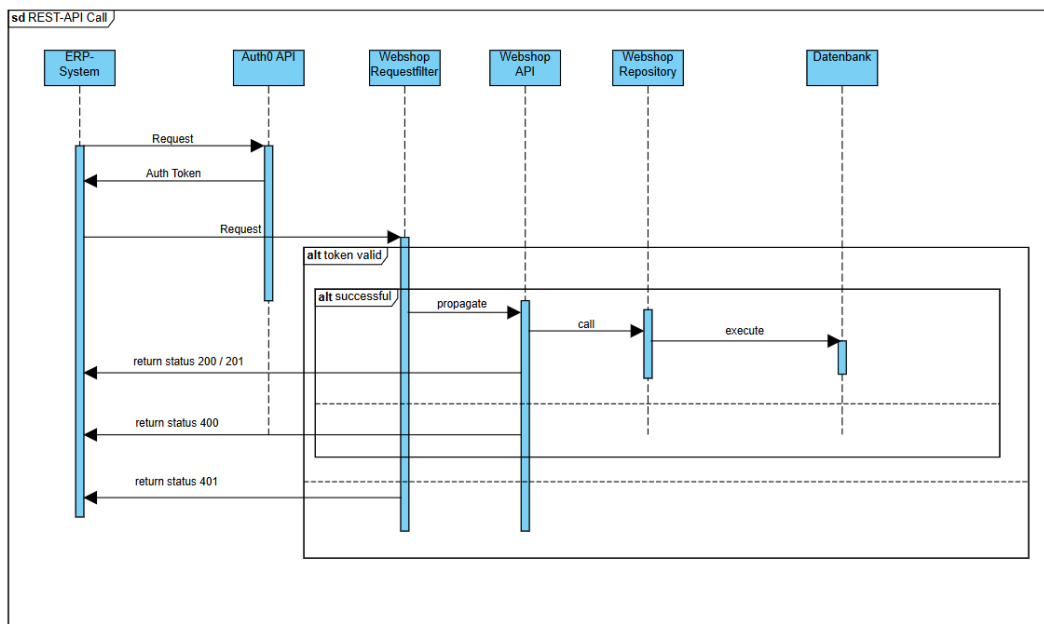


Figure 4: REST-API Call Sequenzdiagramm

Rendern der Artikeldetail Ansicht

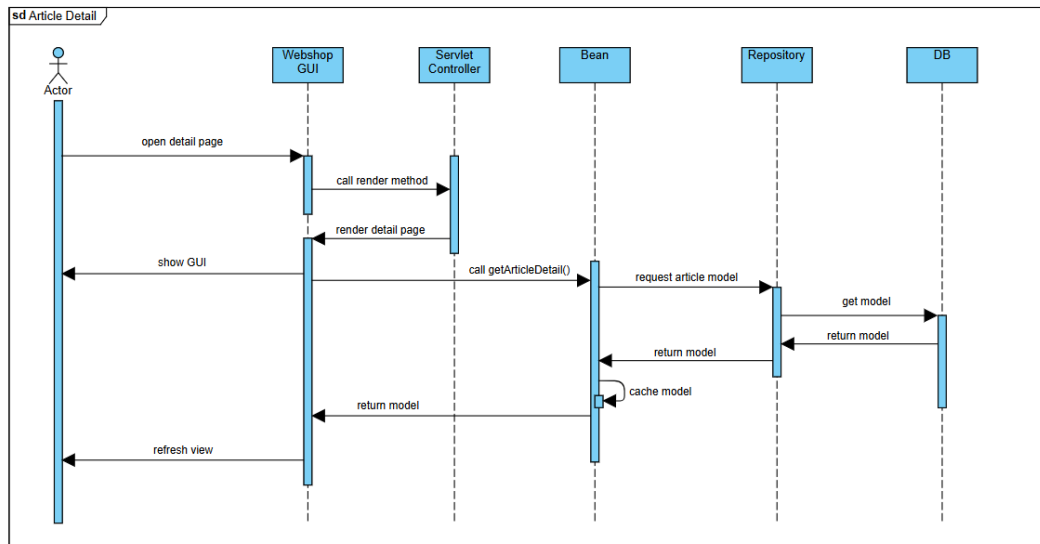


Figure 5: Sequenzdiagramm Artikeldetail Ansicht

Datenbankmodell

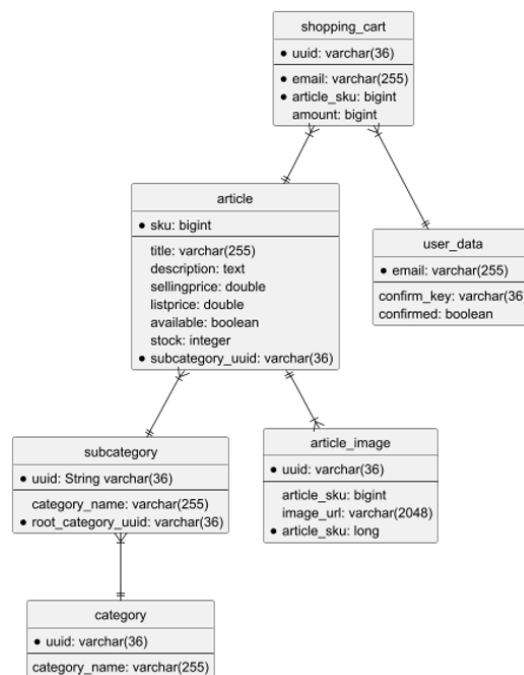


Figure 6: ERM-Diagramm der Datenbank

Verteilungssicht

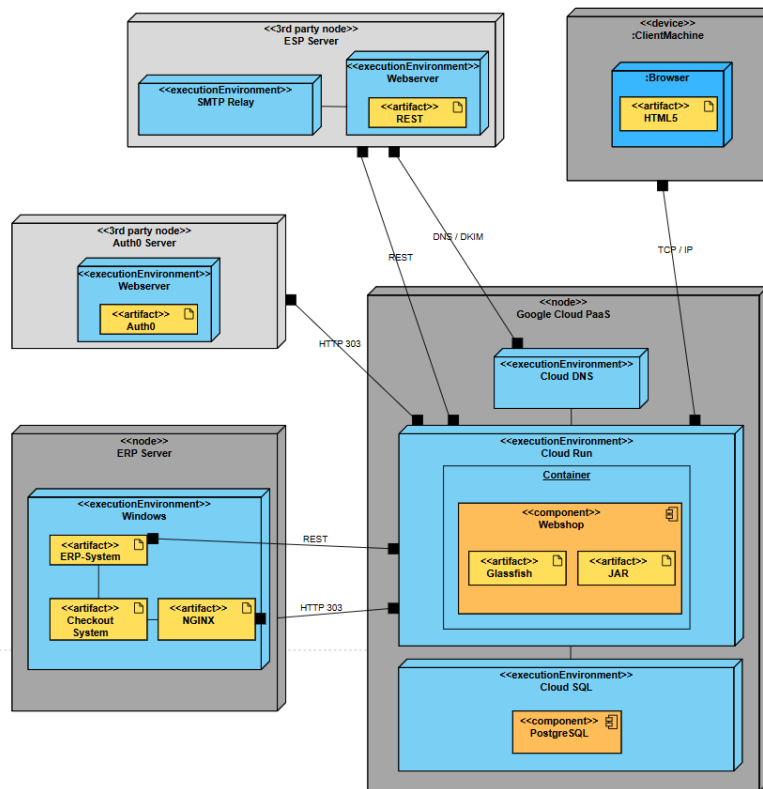


Figure 7: Verteilungsdiagramm produktive Installation

Die Architektur besteht aus zwei Servern dritter Parteien sowie zwei eigenen Servern. Der Versand der Bestätigungsmail findet produktiv über einen Email Service Provider statt. Dazu wird auf die REST-API des Anbieters zugegriffen, die auf einem externen Server zusammen mit dem SMTP-Relay gehostet wird. Das Login funktioniert über Auth0 umgesetzt, das ebenfalls auf einem Third-Party-Server gehostet wird. Der Webshop leitet über einen HTTP Redirect auf die zugehörige URL weiter.

Die Artikel- und Kategoriendaten stammen aus dem ERP-System, das auf einem internen Kundenserver gehostet wird und auf die REST-API des Webshops zugreift. Der Dienst des Checkout-Prozesses ist eine Third-Party Software, die ebenfalls auf dem Kundenserver gehostet und über einen Reverse-Proxy wie NGINX zugänglich gemacht wird. So kann der Webshop auf die entsprechende URL redirecten.

Der Webshop selbst wird auf der Google Cloud mithilfe eines Glassfish Containers betrieben. Die DNS-Verwaltung findet hierbei über Cloud DNS statt. Für die Datenbank wird Cloud SQL mit einer Postgres-Datenbank verwendet. Der Zugriff auf den Webshop erfolgt über den Browser.

Querschnittliche Konzepte

Architektur & Patterns

- Das Projekt wird mit reinem Jakarta EE und Java Server Faces umgesetzt. Sämtliche verwendete Dependencies müssen framework-unabhängig sein.
- Aufgrund der kleinen Grösse des System wird ein klassischer monolithischer Ansatz gewählt, um die Komplexität niedrig zu halten und Ladezeiten zu minimieren.
- Gemäss dem Jakarta-EE Standard wird eine Three-Tier-Architektur aus Datenbank, Backend und Frontend gewählt.
- Die Daten im Frontend werden nach dem MVC-Pattern berechnet und angezeigt

Entwicklung und Testing

- Mit Mockito und JUnit werden die Methoden mit Business-Logik durch Unit-Tests abgesichert. Die Resultate werden mithilfe von Jacoco Reports dokumentiert. Bei jedem Builden der Applikation werden die Tests automatisiert ausgeführt.
- Zur Dokumentation des Codes wird JavaDoc verwendet, die REST-Schnittstellen werden mit Swagger dokumentiert. Aus der Spezifikation wird mittels Github Actions automatisch eine graphische Ansicht gebaut.
- Vor jedem Versionsrelease wird ein manueller, funktionaler Regressionstest durchgeführt, um das korrekte Verhalten für den User sicherzustellen. Zudem wird bei neuen Major-Releases ein Usability-Tests durchgeführt, um eine zufriedenstellende UX sicherstellen zu können.
- Zur Beschleunigung der Entwicklung werden Hilfs-Libraries wie Lombok eingesetzt

Sicherheit

- Um die REST-API abzusichern, wird M2M-Authentifizierung von Auth0 verwendet. Das Login von Benutzern wird ebenfalls mit Auth0 umgesetzt
- Da Benutzer nur auf ihre eigenen Daten zugreifen können, müssen die Daten des Warenkorbs eindeutig einem Benutzer zugeordnet werden können. Deshalb wird dafür ein Login verlangt.

Deployment

- Als Build-Tool wird Maven verwendet. Damit wird die Applikation als JAR-File gebaut.
- Die Applikation läuft als Container auf der Google Cloud. Als Webserver kommt Glassfish zum Einsatz, der das JAR-File als Web-Anwendung zur Verfügung stellt.
- Als Datenbank wird PostgreSQL verwendet, die in der CloudSQL-Umgebung läuft.

Qualitätsanforderungen

Qualitätsbaum

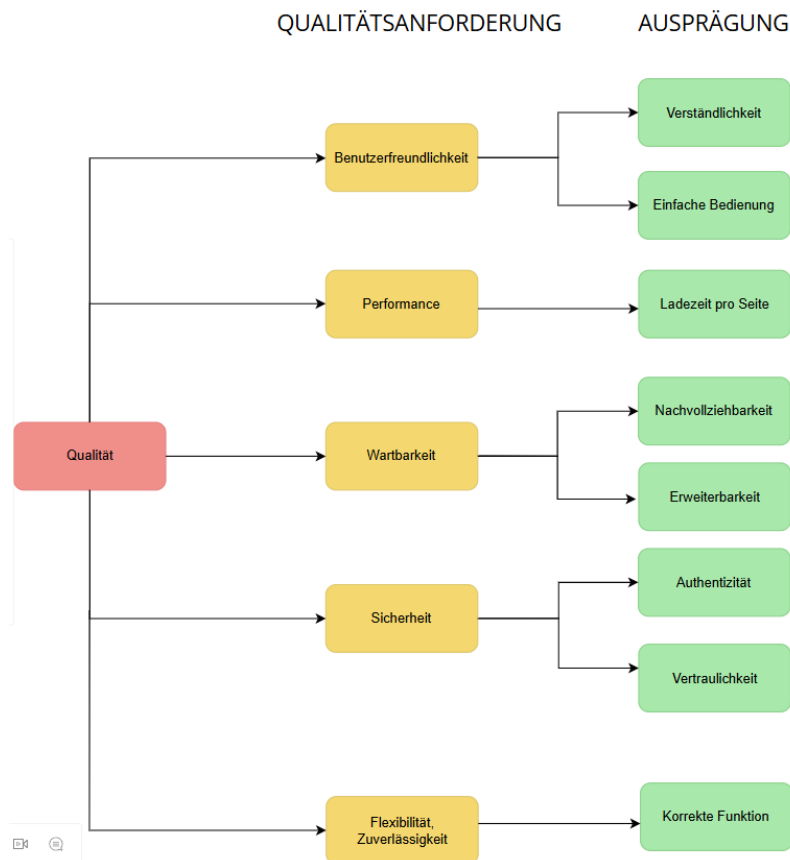


Figure 8

Qualitätsszenarien

- Der Kunde führt eine beliebige Operation im Webshop aus. Die Wartezeit, bis die Operation vollständig abgeschlossen ist, darf bei Normallast ($\leq 90\%$ CPU-Auslastung) maximal 500ms betragen.
- Ein Benutzer versucht, manuell über die REST-API des Webshops Daten zu verändern. Die Anfrage muss abgelehnt werden und die Aktion darf nicht ausgeführt werden.
- Ein Benutzer versucht, auf die Warenkorbdaten eines anderen Benutzers zuzugreifen. Dies darf ohne Logindaten des entsprechenden Benutzers nicht möglich sein.
- Ein Entwickler muss eine Änderung an der Navigation zwischen den Artikelseiten umsetzen. Durch die ausführliche Dokumentation hat er eine niedrige Lernkurve und implementiert die Anforderung in min. 50% der Fälle korrekt, sodass keine nachträglichen Bugfixes mehr nötig sind.
- Eine neue Quelle für Stammdaten muss an den Webshop angeschlossen werden. Durch die ausführliche Dokumentation der API kann dies in min. 90% der Fälle durch die Entwickler der Quelle selbstständig umgesetzt werden, ohne Unterstützung der Webshop-Entwickler.

- Ein unerfahrener Mitarbeiter setzt eine Änderung am Webshop um und führt dabei einen Bug ein. Die entsprechenden Unit-Tests schlagen fehl und sorgen dafür, dass der fehlerhafte Code nicht in der Produktion landet. Pro neuer Version darf es max. einen kritischen Bug in der Produktion geben.

Risiken und technische Schulden

Folgende Risiken und technische Schulden existieren im Projekt, geordnet nach Priorität:

- Anbindung an geeignetes Checkout-System: Aktuell ist der Webshop nur bis zum Warenkorb umgesetzt, für einen produktiven Einsatz muss der Checkout-Prozess noch angebunden werden
- Skalierbarer Maildienst: Aktuell verwendet der Webshop den normalen SMTP-Server von Gmail. Für einen flächendeckenden Einsatz sollte ein Email Service Provider wie MailJet eingesetzt werden.
- Verwendung von Frontend-Libraries: Aktuell ist das Frontend eher einfach gehalten und basiert auf standardisierten HTML-Elementen. In einer späteren Version kann das GUI durch den Einsatz von Frontend-Libraries modernisiert werden.
- Skalierung: Mit der Zunahme der Datenmengen und der gleichzeitigen Benutzer muss das Deployment Load Balancing unterstützen. Aktuell gibt es nur eine Instanz der Datenbank und Applikation.
- Aktuell lässt sich der Webshop nur lokal über Docker / Podman betreiben, ein Deployment auf Google Cloud muss noch umgesetzt werden. Dies ist notwendig, um die nötigen Werkzeuge zur Skalierung und Ausfallsicherheit zur Verfügung zu haben.

Abnahmekriterien

- Vervollständigung der Funktionalität gem. technischer Schulden (vgl. technische Schulden)
- Alle Unit-Tests laufen erfolgreich durch
- Alle Abhängigkeiten haben die neuste kompatible Version
- Sämtliche Qualitätsanforderungen sind erfüllt (vgl. Qualitätsziele / Qualitätsanforderungen)
- Die Dokumentation ist aktualisiert und entspricht dem gegenwärtigen Stand
- Die Applikation kann erfolgreich auf Google Cloud deployed und mit einer eigenen Domain versehen werden.

Weitere Erkenntnisse aus der Programmierung

Mit diesem Projekt konnte das Entwicklungsteam seine Fertigkeiten im Umgang mit der Entwicklung einer neuen Applikation von Grund auf verbessern. Zudem konnten erste Erfahrungen mit neuen Technologien wie Email Service Providern (ESP) und Jacoco gewinnen. Bei anderen Technologien konnte das Wissen aufgefrischt werden, beispielsweise mit Mockito, Auth0 und Swagger. Das Entwicklungsteam lernte so auch, selbstständig Technologientendenzen zu treffen und mithilfe der Dokumentation effizient umzusetzen.

Das Entwicklungsteam folgt bei der Entwicklung grundsätzlich streng den definierten Programmierrichtlinien. Durch die Standardisierung von Architektur und Package-Struktur konnte diese Applikation auf Basis eines älteren Projektes umgesetzt werden, das entsprechend angepasst und erweitert wurde. Damit lässt sich nachweisen, dass der zusätzliche Aufwand für die Erfüllung der Richtlinie sich an anderer Stelle auszahlt.

Durch die positiven Erfahrungen mit JUnit und Jacoco wurde die Programmierrichtlinien im Zuge

dieses Projekts um obligatorische Unit-Tests erweitert. Zuvor war es den Entwicklern freigestellt, ob zusätzlich zu manuellen Funktionstest auch automatisierte Unit-Tests eingesetzt wurden.

Glossar

Begriff	Definition
Anbindung externer Systeme	eine andere Applikation interagiert greift autorisiert auf die REST-API des Webshops zu
Volltextsuche	es wird nach Artikel gesucht, deren Titel den Suchbegriff enthalten; am Anfang, in der Mitte oder am Ende
Bestätigungsmail	eine E-Mail, die einen Link enthält, den man öffnen muss; damit bestätigt man, dass man die E-Mail erhalten und damit Zugriff auf das E-Mail Postfach hat.
Business-Logik	Funktionen des Webshops die nicht standardisiert sind. Nicht zur Business-Logik zählen beispielsweise Mathematik- oder CRUD-Funktionen.
Mail-Server	Server, der angefragt werden kann, um Mails zu verschicken
Checkout-Prozess	umfasst Eingabe von Lieferadresse und Zahlungsmethode
Login Callback	Bevor auf das Login-Formular von Auth0 umgeleitet wird, wird definiert, was ausgeführt werden soll, wenn der Request anschliessend wieder zurück auf den Webshop geht. Dies ist die Callback-Funktion.
Servlet-Route	Route, die verwendet werden, um Seiten oder deren Daten zu laden und anzuzeigen
Request Filter	ein Filter, der bei jedem Request überprüft, ob dieser zugelassen werden soll
JWT	Eine verschlüsselte Zeichenkette, die die Authentifizierung und Autorisierung eines Benutzers bestätigt.